



KAZAKH-BRITISH
TECHNICAL
UNIVERSITY

JSC «Kazakh-British Technical University»

School of IT and Engineering



Azamat Imanbayev

« 19 »

2026

SYLLABUS

Discipline: Advanced Software Paradigms

Number of credits: 3

Course code: CSDA 5101

Term: Fall 2026

Instructor's full name: Bryan A. Jones

Personal Information about the Instructor	Time and place of classes		Contact information
	Lessons	Office Hours	e-mail
Associate Professor of SITE	According to the schedule	By appointment: https://bjones.youcanbook.me/	b.jones@kbtu.kz

COURSE DURATION: 3 credits, 15 weeks, 45 class hours

MS Teams code: 6s7365d

GENERAL COURSE AIMS:

This hands-on, project-focused course develops students' programming skills by changing the way they think about writing software and firmware. Students learn to write code while explaining their ideas so that others can understand, maintain, and contribute to their work, emphasizing interleaved documentation, distributed version control, and social coding practices that support code longevity and effective collaboration.

COURSE DESCRIPTION

Techniques for software development and embedded systems, with an emphasis on interleaved documentation for code longevity. Students write software while learning literate-programming techniques so that their documentation and code evolve together.

COURSE OBJECTIVES

The main objective of this course is to prepare students to write high-quality, well-documented software by applying literate programming techniques, interleaving documentation with source

code, and using distributed version control (DVCS) and social coding practices (e.g., GitHub pull requests and code review) to coordinate collaborative software development.

COURSE OUTCOMES

After successfully completing this course, students will be able to:

1. Discuss various approaches to literate programming and demonstrate knowledge of markup formats used to intermingle source code with structured comments; apply modern literate programming tools.
2. Demonstrate proficiency in literate programming methods by developing projects in which documentation is interleaved with source code.
3. Utilize a distributed version control system to record the evolution of software using branches and commits; employ merge/rebase strategies to incorporate branches from other developers; describe the locations in which a DVCS stores its data and the methods used to move data between these locations.
4. Employ social coding techniques to submit a pull request and to write and respond to code reviews; apply software management techniques to coordinate decentralized development across multiple teams.

COURSE POST REQUISITES

LITERATURE

1. Rizwan Qureshi, *Software Engineering Paradigm: Principles and Practices*, VDM Verlag Dr. Müller, 2010
2. Maurizio Gabbrielli and Simone Martini, *Programming Languages: Principles and Paradigms (Undergraduate Topics in Computer Science)*, Springer, 2010 edition.
3. D. E. Knuth, "Literate Programming," *CSLI Lecture Notes*, Stanford, 1992.
4. N. Ramsey, "Literate Programming Simplified," *IEEE Software*, vol. 11, no. 5, 1994.
5. S. Chacon and B. Straub, *Pro Git*, 2nd ed., Apress, 2014.
6. J. Loeliger and M. McCullough, *Version Control with Git*, 2nd ed., O'Reilly, 2012.
7. R. C. Martin, *Clean Code: A Handbook of Agile Software Craftsmanship*, Prentice Hall, 2008.
8. A. Hunt and D. Thomas, *The Pragmatic Programmer*, 2nd ed., Addison-Wesley, 2019.
9. G. Wilson et al. (eds.), *Beautiful Code*, O'Reilly, 2007.
10. M. Fowler, *Refactoring: Improving the Design of Existing Code*, 2nd ed., Addison-Wesley, 2018.
11. A. Oram and G. Wilson (eds.), *Making Software: What Really Works, and Why We Believe It*, O'Reilly, 2010.
12. K. Fogel, *Producing Open Source Software*, O'Reilly, 2005.
13. GitHub, Inc., *GitHub Docs: Collaborating with Pull Requests*, docs.github.com.
14. B. W. Kernighan and R. Pike, *The Practice of Programming*, Addison-Wesley, 1999.
15. M. Rüping, *Agile Documentation: A Pattern Guide to Producing Lightweight Documents for Software Projects*, Wiley, 2003.

COURSE ASSESSMENT CRITERIA

Assessment occurs continuously throughout the course. The evaluation will be based on the levels of (maximums in %):

Type of activity	Final scores
Attendance / participation	5%
Practice (Homework)	25%
TSIS (Projects — GitHub pull requests)	30%
Final exam (Final project defense, weeks 16–17)	40%
Total	100%

Note: this course is project-based and does not use a traditional written final exam; the "Final exam" row above corresponds to the Final Project defense/demonstration in Weeks 16–17.

TASKS

for practice (SIS — weekly homework, submitted via Teams/GitHub)

Week	SIS	Cost (in points)
8	<i>Specification design</i>	10
9	<i>Specification refinement and initial implementation</i>	10
10	<i>Specification finalization, implementation finalization</i>	10
	Total	30

COURSE CALENDAR (Tentative)

Week	Topic	Lectures (hours)	Practice (hours)
1	Writing a spec and tests	3	
2	Review of literate programming	3	
3	Introduction to social coding: git	3	
4	Introduction to social coding: GitHub	3	
5	CodeChat Editor (CCE) use: documentation as design	3	
6	Integrating CCE, Git, and social coding with LLM use	3	
7	Data structures as design; LLM techniques	3	
8	Specification design for project 1	3	
9	Specification refinement and initial implementation for project 1	3	

Week	Topic	Lectures (hours)	Practice (hours)
10	Specification finalization, implementation finalization for project 1	3	
11	Project 1 defense	3	
12	Specification design for final project	3	
13	Specification refinement for final project	3	
14	Initial implementation for final project	3	
15	Specification finalization, implementation finalization for final project	3	
16–17	Final Project defense		

Course assessment schedule

No	Assessment criteria	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	Total
1	Attendance / participation	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	5%
2	Practice	*	*	*	*	*	*	*									25%
3	SIS								*	*	*	*	*	*	*	*	30%
4	Final project defense																40%

Class sessions — will be a mixture of information, discussion, and practical application of skills.

Practice — will reinforce students' knowledge through practical application of lectured material and homework assigned and submitted through Teams and GitHub.

In-class assessment — will prepare students for their mid-term and final assessment and identify the competence level they have achieved on a related subject matter, the aim being to diagnose potential discrepancies in students' understanding and performance in order to make specific adjustments to the course content and procedures and/or to assign additional assignments to certain individuals or the whole group.

Home assignments (Practice) — will consolidate the concepts and materials covered during in-class activities, help students expand their understanding through diverse background resources, and practice specific skill areas; they will also develop students' ability to work individually in exploring and examining related issues.

SIS (Student Independent Study) — comprises a significant project in which students contribute bug fixes or feature additions, in the form of accepted pull requests, to large, ongoing software development projects or to a newly-defined application. Students are expected to become quickly conversant in new languages and libraries in order to provide meaningful contributions to these projects. Assistance and advice will be provided by the instructor during office hours.

Final project defense — students demonstrate their final project, in which documentation is fully interleaved with source code per literate programming requirements, and defend their design and implementation choices before the instructor.

Grading policy:

Intermediate attestations (in Week 8 and Week 14) join topics of all lectures, practice, laboratories, SIS, TSIS, and materials for reading discussed up to the time of attestation. Maximum number of points within attendance, activity, SIS, TSIS, and laboratories for each attestation is 30 points.

Final exam (Final Project defense) joins and generalizes all course materials and is conducted as a project demonstration and defense in Weeks 16–17. Maximum number of points is 40. At the end of the semester, you receive an overall total grade (a summarized index of your work during the semester) according to the conventional KBTU grade scale.

ATTENTION!

- 1) If a student misses more than 30% of lessons, the student receives an «F (Fail)» grade;
- 2) If for two attestations a student receives 29.4 points or fewer, this student is not admitted to the final exam and receives an «F (Fail)» grade for the whole course;
- 3) If a student receives 9.4 points or fewer on the final exam, then regardless of the points received for the two attestations, the student receives an «F (Fail)» grade in whole;

In the case of missing or being late for the final exam without a plausible reason, regardless of the points received for the two attestations, the student receives an «F (Fail)» grade in whole.

ACADEMIC POLICY:

1. Cheating, duplication, falsification of data, and plagiarism are not permitted under any circumstances.
2. Students must participate fully in every class. While attendance is crucial, merely being in class does not constitute "participation." Participation means reading the assigned materials and coming to class prepared to ask questions and engage in discussion.
3. Students are expected to take an active role in learning (the instructor will provide the information and guidelines to do this).
4. Students must come to class on time.
5. Students are responsible for making up any work missed.
6. Make-up tests in case of absence will not normally be allowed.
7. Mobile phones must always be switched off in class.
8. Students should always show tolerance, consideration, and mutual support toward other students.
9. Generative AI Use: This course is designated Generative AI Use Level D (AI-Integrated). See the "Generative AI Use Policy" section below for full details and requirements.

GENERATIVE AI USE POLICY

AI-Integrated. Generative AI may be used as a regular tool throughout the coursework, including generation of text, code, ideas, analyses, or solutions. Students are responsible for reviewing, adapting, verifying, and taking full responsibility for the final work. Example: "Write a Java implementation of this class according to these requirements."

Course-specific requirements for Generative AI use:

- Students may use Generative AI tools (e.g., ChatGPT, Claude, GitHub Copilot) as a regular part of coursework, including generation of code, documentation, ideas, and analyses. Students remain fully responsible for reviewing, adapting, verifying, and taking full responsibility for all submitted work.
- Students must give credit for and cite any AI-generated material according to standard rules of documentation, including in-text citations, quotations, and references.
- GitHub commits produced using AI (which should be the majority of commits in this course) should include a "Co-Authored-By:" line, for example: "Co-Authored-By: Claude Opus 5 <noreply@anthropic.com>".

GRADING SCALE

Grade		Achievement percentage	Assessment criterion
«Excellent»	A	95-100%	This grade is given when the student: demonstrated a complete understanding of the course material; did not make any errors or inaccuracies; completed control and laboratory work in a timely and correct manner, and submitted reports on them; demonstrated original thinking; submitted control quizzes on time and without any errors; completed homework assignments; engaged in research work; independently used additional scientific literature in studying the discipline; was able to independently systematize the course material.
	A-	90-94%	
«Good»	B+	85-89%	This grade is given when the student: has mastered the course material at no less than 75%; did not make gross errors in responses; timely completed control and laboratory work and submitted them without fundamental remarks; correctly completed and timely submitted control tests and homework assignments without fundamental remarks; utilized additional literature as indicated by the instructor; engaged in research work, made non-fundamental errors, with fundamental errors corrected by the student themselves; managed to systematize the course material with the help of the instructor.
	B	80-84%	
	B-	75-79%	
	C+	70-74%	
«Satisfactory»	C	65-69%	This grade is given when the student: has mastered the course material no less than 50%; required assistance from the instructor when completing control and laboratory work, homework assignments; made inaccuracies and non-fundamental errors when submitting control tests; did not demonstrate activity in research work, relied solely on the educational literature indicated by the instructor; experienced more difficulty in systematizing the material.
	C-	60-64%	
	D+	55-59%	
	D	50-54%	

Instructor

Associate Professor of SITE



Bryan A. Jones

Academic Head of Industrial Technologies, Ph.D



Timur I. Samigulin

Minutes No. 1 of the School of Information Technology and Engineering meeting, August 18, 2026.